

API SECURITY RISK ANALYSIS REPORT

****Prepared by:** MUSTAPHA ALIYU UMAR

****Date:** 5/18/2026

****Target API:** JSONPlaceholder (<https://jsonplaceholder.typicode.com>)

****Testing Tools:** Postman, Browser DevTools

1. EXECUTIVE SUMMARY

I conducted a security risk analysis of the JSONPlaceholder test API. The assessment identified ***5 security risks***, including ***3 critical authentication/authorization issues***.

Severity	Count	Risks Identified
 High	3	No authentication, No authorization, DELETE request without authentication
 Medium	1	Excessive data exposure
 Low	1	No rate limiting observed

****Key Finding:**** The API accepts all requests (GET, POST, PUT, DELETE) without any form of authentication or authorization. Any user can read, create, update, or delete any data.

2. TESTING METHODOLOGY

Tools Used

Tool	Purpose
Postman	Send API requests (GET, POST, DELETE)
Browser DevTools	Inspect headers, tokens, and responses

Endpoints Tested

Endpoint	Method	Tested
/posts	GET	✓
/posts/1	GET	✓
/posts	POST	✓
/posts/1	DELETE	✓
/posts?userId=1	GET	✓

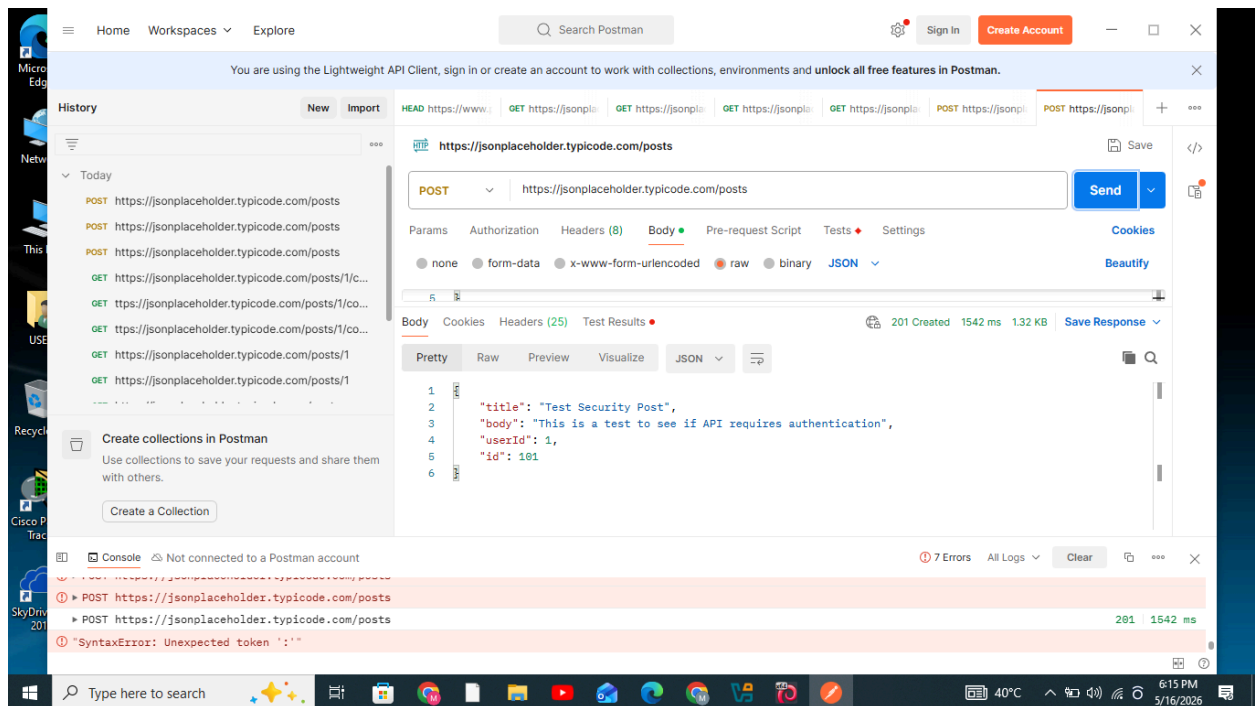
3. DETAILED FINDINGS

FINDING #1: No Authentication Required (HIGH RISK)

Location: All endpoints

Method Tested: POST /posts

Evidence (Postman Screenshot):



****Technical Description:**

The API accepted a POST request to create a new post without any API key, bearer token, or authentication header.

****Business Translation:**

This is like having a public mailbox where anyone can put letters in, take letters out, or throw letters away. No one needs a key. An attacker could flood the system with fake data or delete real data.

****Remediation:**

```markdown

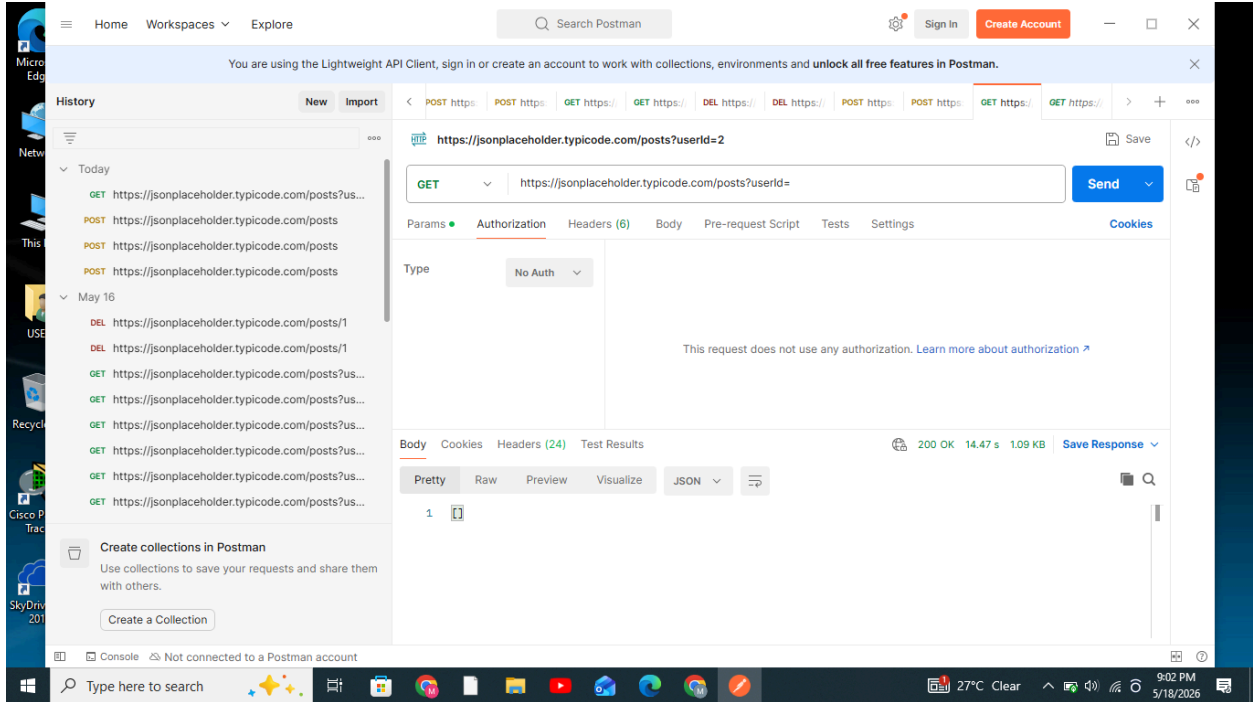
- Implement API key authentication
- Use Bearer tokens (JWT)
- Require authentication for all write operations (POST, PUT, DELETE)

## **FINDING #2: No Authorization Controls (HIGH RISK)**

Location: GET /posts?userId=X

Method Tested: GET with different userId parameters

Evidence (Postman Screenshot):



#### Technical Description:

The API returns posts from ANY userId without checking if the requester owns that data.

#### Business Translation:

If this API was used by a social media app, any user could read ANY other user's private posts. A competitor could download all your customers' data.

#### Remediation:

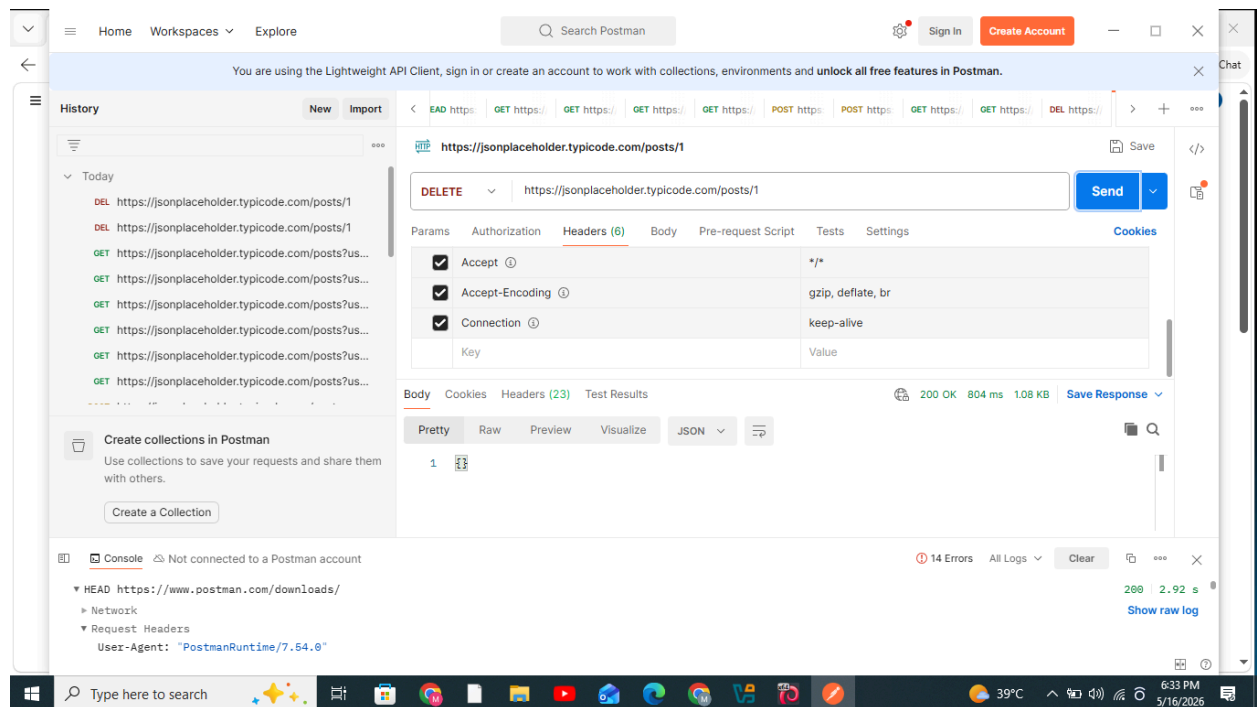
- Implement access control checks
- Only return data belonging to the authenticated user
- Use user ID from authentication token, not from URL parameter

### FINDING #3: DELETE Without Authentication (HIGH RISK)

Location: DELETE /posts/1

Method Tested: DELETE

## Evidence (Postman Screenshot):



## Technical Description:

The API accepted a DELETE request without any authentication, successfully removing resource ID 1.

## Business Translation:

Anyone can delete data from the system. An attacker could wipe out your entire database.

## Remediation:

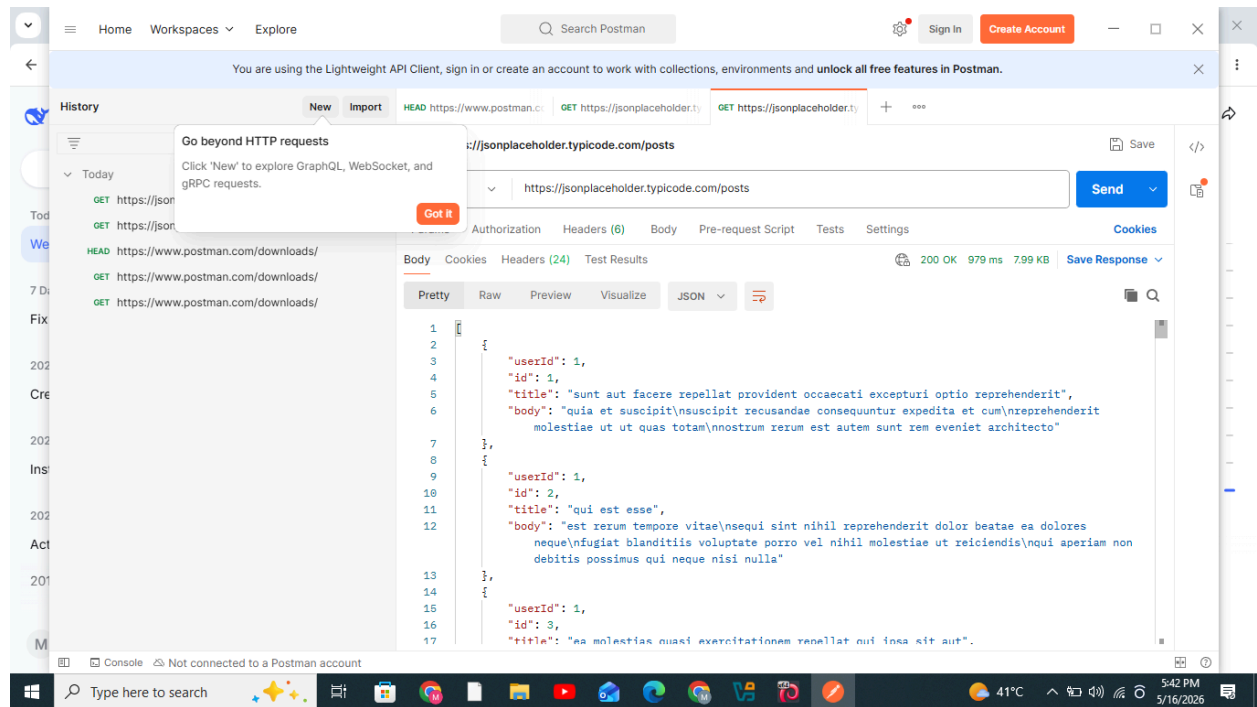
- Require authentication for DELETE operations
- Implement soft delete (mark as deleted, don't remove)
- Add audit logging for all delete operations

## FINDING #4: Excessive Data Exposure (MEDIUM RISK)

Location: GET /posts

Method Tested: GET

Evidence (Postman Screenshot):



Technical Description:

The API returns all fields (userId, id, title, body) without filtering sensitive data.

Business Translation:

The API sends more information than needed. If a mobile app only needs post titles, sending full content wastes bandwidth and exposes more data than necessary.

Remediation:

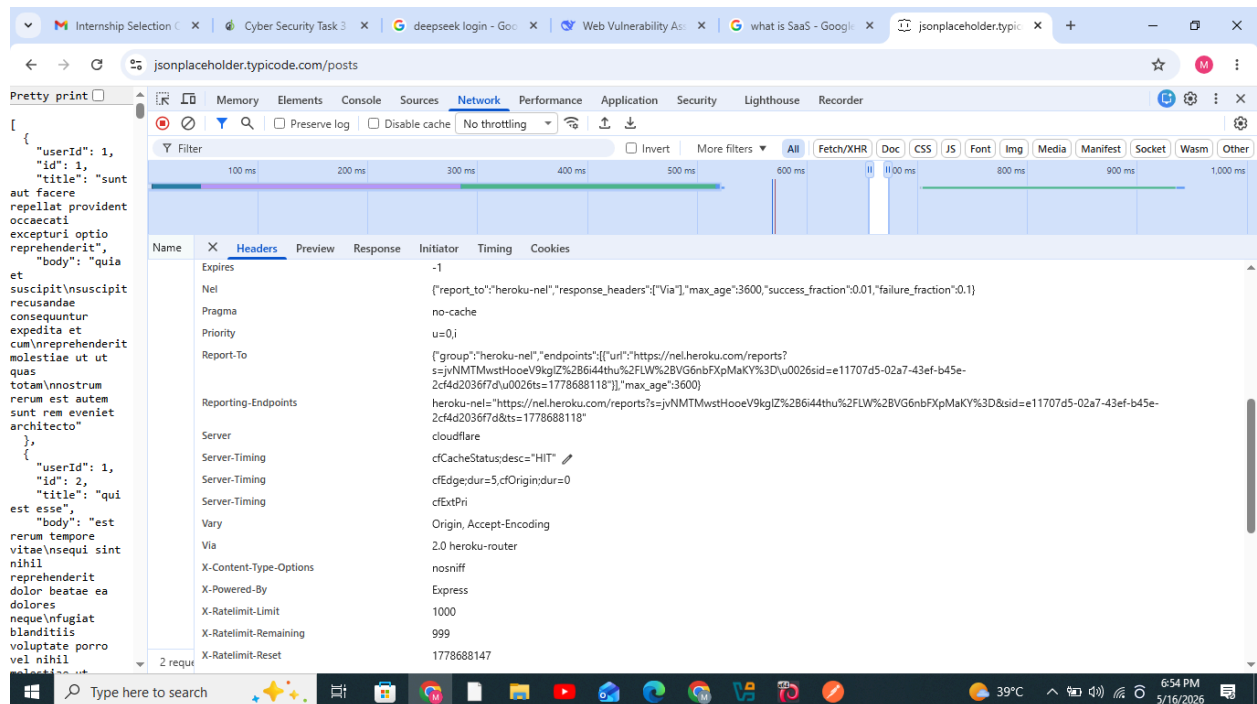
- Implement field filtering (only return requested fields)
- Use GraphQL for precise data requests
- Don't return internal fields to clients

## FINDING #5: No Rate Limiting Observed (LOW RISK)

Location: All endpoints

Method Tested: Multiple rapid GET requests

Evidence: All requests succeeded within seconds



Technical Description:

The API responded to multiple rapid requests without any throttling or rate limit errors.

Business Translation:

An attacker could hammer the API with thousands of requests per second, potentially causing a denial of service or increasing your cloud costs.

Remediation:

- Implement rate limiting (e.g., 100 requests per minute)
- Use API gateway with throttling
- Return 429 (Too Many Requests) when limit exceeded

## 4. RISK SUMMARY TABLE

| Risk                    | Severity                                                                                 | Impact                            | Remediation                |
|-------------------------|------------------------------------------------------------------------------------------|-----------------------------------|----------------------------|
| No Authentication       |  HIGH   | Anyone can modify/delete data     | Add API keys + tokens      |
| No Authorization        |  HIGH   | Anyone can access any user's data | Implement access control   |
| DELETE without Auth     |  HIGH   | Data can be deleted by anyone     | Require auth + soft delete |
| Excessive Data Exposure |  MEDIUM | Unnecessary data sent to clients  | Implement field filtering  |
| No Rate Limiting        |  LOW  | Potential DoS vulnerability       | Add request throttling     |

## 5. REMEDIATION PRIORITIES

### IMMEDIATE (24 hours):

- Require authentication for all POST, PUT, DELETE requests
- Implement API key validation
- Remove ability to access other users' data via userId parameter

### SHORT TERM (1 week):



- Add rate limiting (100 requests/minute)
- Implement field filtering
- Add audit logging for all write operations

### LONG TERM (1 month):

- Migrate to JWT-based authentication
- Implement role-based access control (RBAC)
- Regular security testing of API endpoints

## 6. PREVENTION GUIDELINES FOR DEVELOPERS

### API Security Checklist:

| Check                                      | Why                                      |
|--------------------------------------------|------------------------------------------|
| ✓ Require authentication for all endpoints | Prevents unauthorized access             |
| ✓ Validate user permissions                | Ensures users access only their own data |
| ✓ Implement rate limiting                  | Prevents DoS attacks                     |
| ✓ Validate all input                       | Prevents injection attacks               |
| ✓ Use HTTPS only                           | Encrypts data in transit                 |
| ✓ Log all access                           | Enables incident investigation           |

## 7. TOOLS USED

| Tool             | Purpose                                  |
|------------------|------------------------------------------|
| Postman          | API endpoint testing (GET, POST, DELETE) |
| Firefox DevTools | Request/response inspection              |
| Google Docs      | Report documentation                     |

## 8. APPENDIX: Screenshot Evidence

- Screenshot 1: GET /posts - Successful response showing all posts
- Screenshot 2: GET /posts/1 - Single post response
- Screenshot 3: POST /posts - Creation without authentication
- Screenshot 4: GET /posts?userId=1 vs userId=2 - No authorization
- Screenshot 5: DevTools Network tab - No auth headers visible

***End of Report***